

Project Development Phase

Code-Layout, Readability and Reusability

Date	02 July 2026
Team ID	
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

This document evaluates the quality of the OptiCrop codebase in terms of its structure, readability, and reusability. The project separates model training (train_model.py) from the web application (app.py), with the trained model persisted as model.pkl so it does not need to be retrained on every run.

Code Layout Checklist

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	4-space PEP8-style indentation used consistently in app.py and train_model.py
2	Proper File Structure	Files and folders are logically organized	Yes	Clear separation: app.py (web app), train_model.py (training), model.pkl (artifact), requirements.txt (dependencies)
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	N, P, K, temperature, humidity, ph, rainfall directly match the domain (soil/climate parameters)

4	Function / Method Names	Functions are descriptively named	Partial	app.py is primarily a linear Streamlit script; core logic could be refactored into named functions (e.g., predict_crop()) for better testability
5	Code Comments	Inline and block comments explain logic	Yes	train_model.py has step-by-step comments (Step 1–8); app.py has a header comment explaining run instructions and dependencies
6	Modular Design	Code is split into reusable functions/modules	Partial	Training and inference are modularized into separate files; app.py itself could be further split into UI and prediction modules
7	No Redundant Code	Duplicate or unused code is removed	Yes	No duplicate logic found across the two files
8	Error Handling	Exceptions and errors are handled gracefully	Yes	app.py wraps model loading in try/except (FileNotFoundError) and prediction in try/except (generic Exception) with user-facing error messages

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High/Medium/Low)
1	model.pkl (trained Random Forest)	scikit-learn (pickle)	Loaded once by app.py; can be reused by any future script, API, or notebook without retraining	High
2	train_model.py	Python / pandas / scikit-learn	Can be re-run on updated datasets to regenerate model.pkl	High
3	Prediction block in app.py	Python / pandas	Core predict + top-3 suggestion logic; reusable if extracted into a function	Medium

Overall Code Quality Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clean, minimal, logically split between training and serving
Readability	4	Descriptive domain-based variable names throughout
Reusability	4	Model artifact and training script are independently reusable
Documentation / Comments	3	Present but could be expanded with docstrings on any future functions
Overall Score	4	Solid for a small single-page ML application